

Programación 4

Informe del Modelo de Diseño

Grupo 83 Integrantes

Fausto Britos
Ana Hackembruch
Francisco Martinez
Franco Panfilo
Daniel Rocha

CI: 5.576.188-5
CI: 5.309.116-7
CI: 5.476.736-1
CI: 5.727.467-2
CI: 5.520.396-0

Docente: Antonio Mautone

1 Realización de Casos de Uso

1.1 Estructura

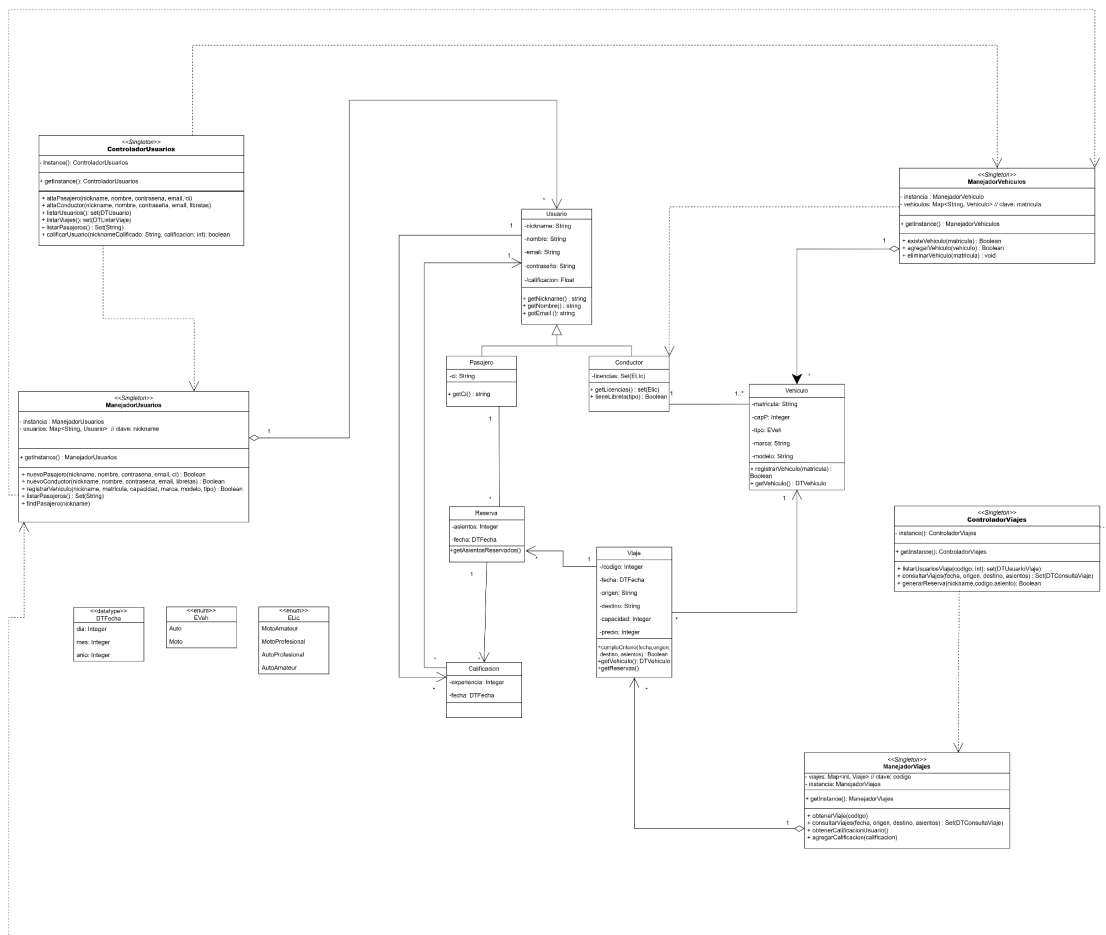


Figura 1.1: Diagrama de Clases de Diseño.

1.2 Interacciones

1.2.1 Diagramas de comunicación de altaPasajero

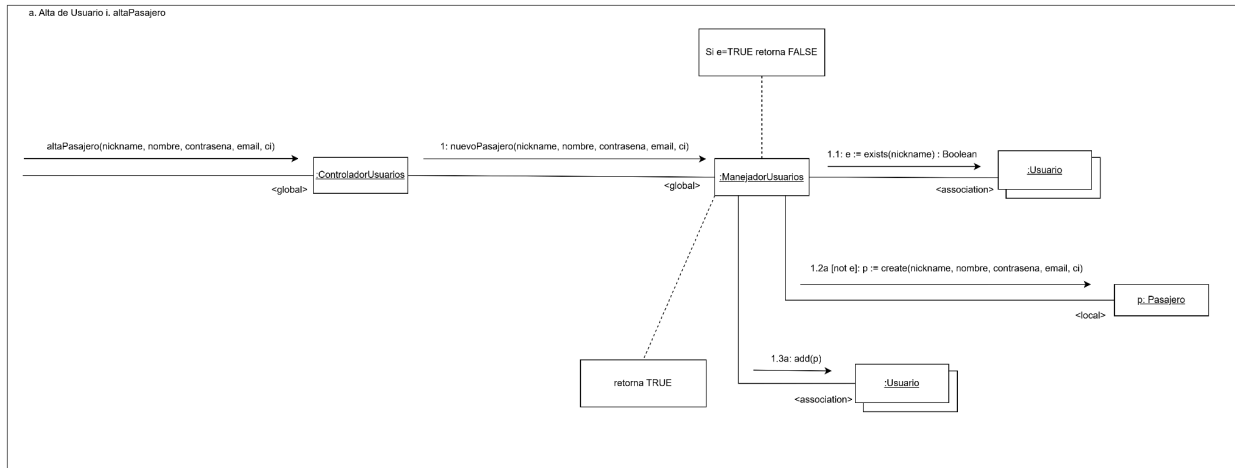


Figura 1.2: Diagrama de comunicación para dar de alta un pasajero.

1.2.2 Diagramas de comunicación de altaConductor

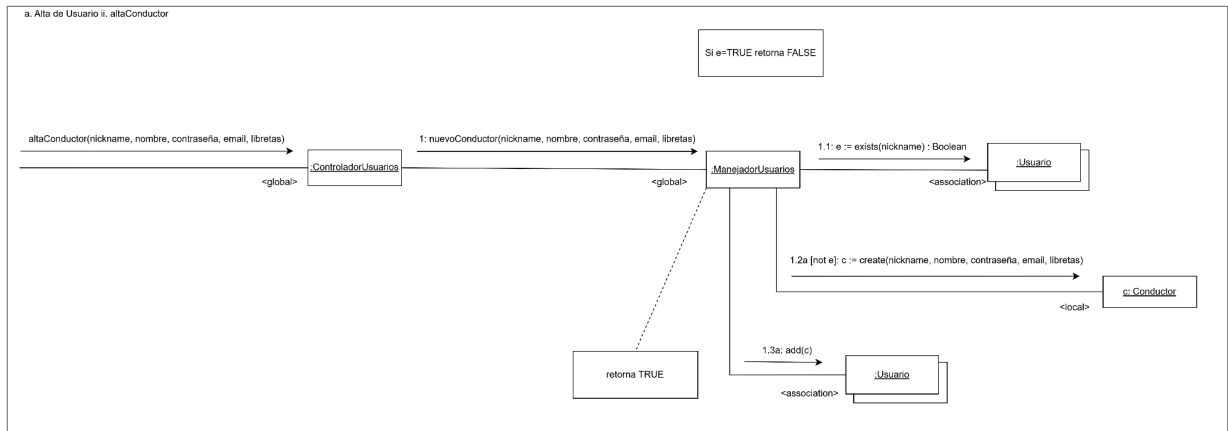


Figura 1.3: Diagrama de comunicación para dar de alta un conductor.

1.2.3 Diagramas de comunicación de registrarVehículo

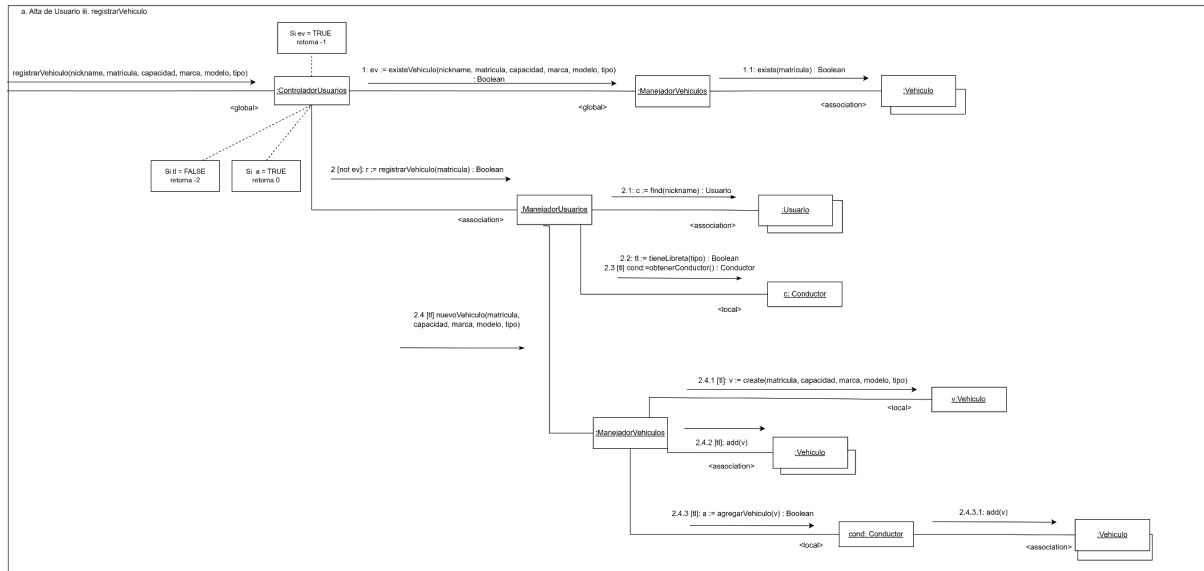


Figura 1.4: Diagrama de comunicación para dar de alta un vehículo de un conductor.

1.2.4 Diagramas de comunicación de listarPasajeros

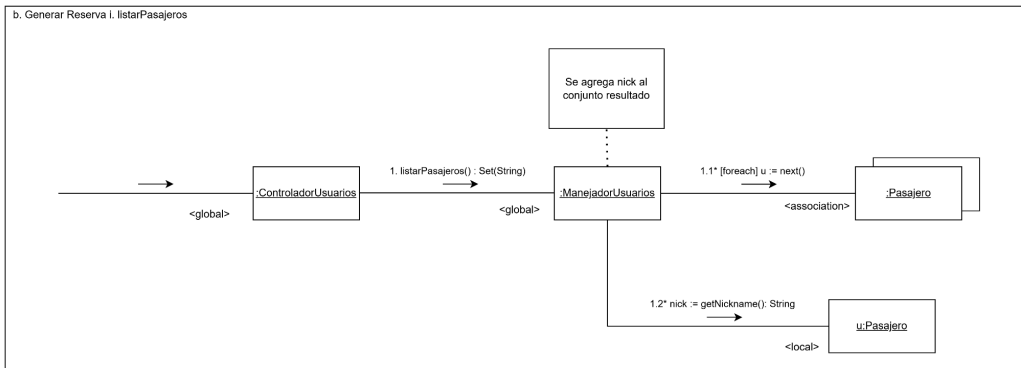


Figura 1.5: Diagrama de comunicación para listar los pasajeros existentes.

1.2.5 Diagramas de comunicación de consultarViajes

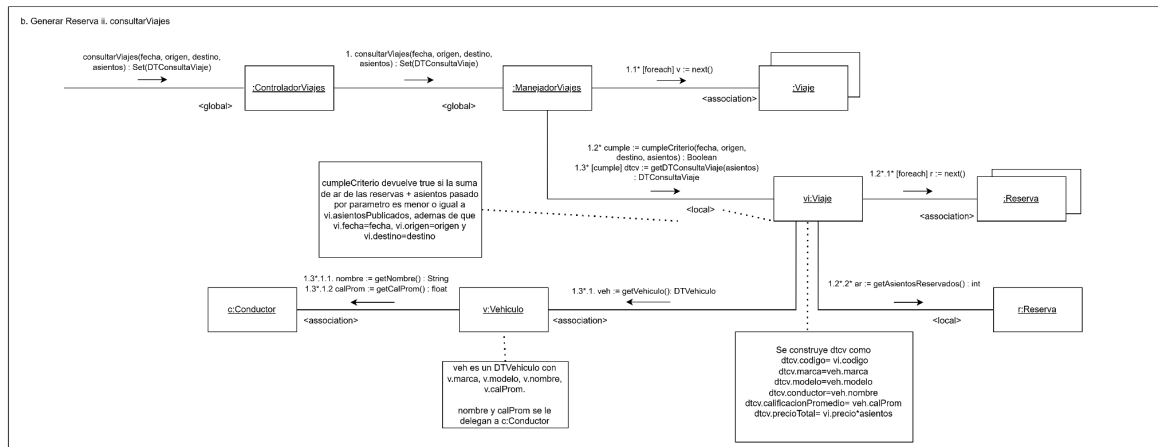


Figura 1.6: Diagrama de comunicación para consultar los viajes de una fecha, con cierto origen destino y asientos.

1.2.6 Diagramas de comunicación de generarReserva

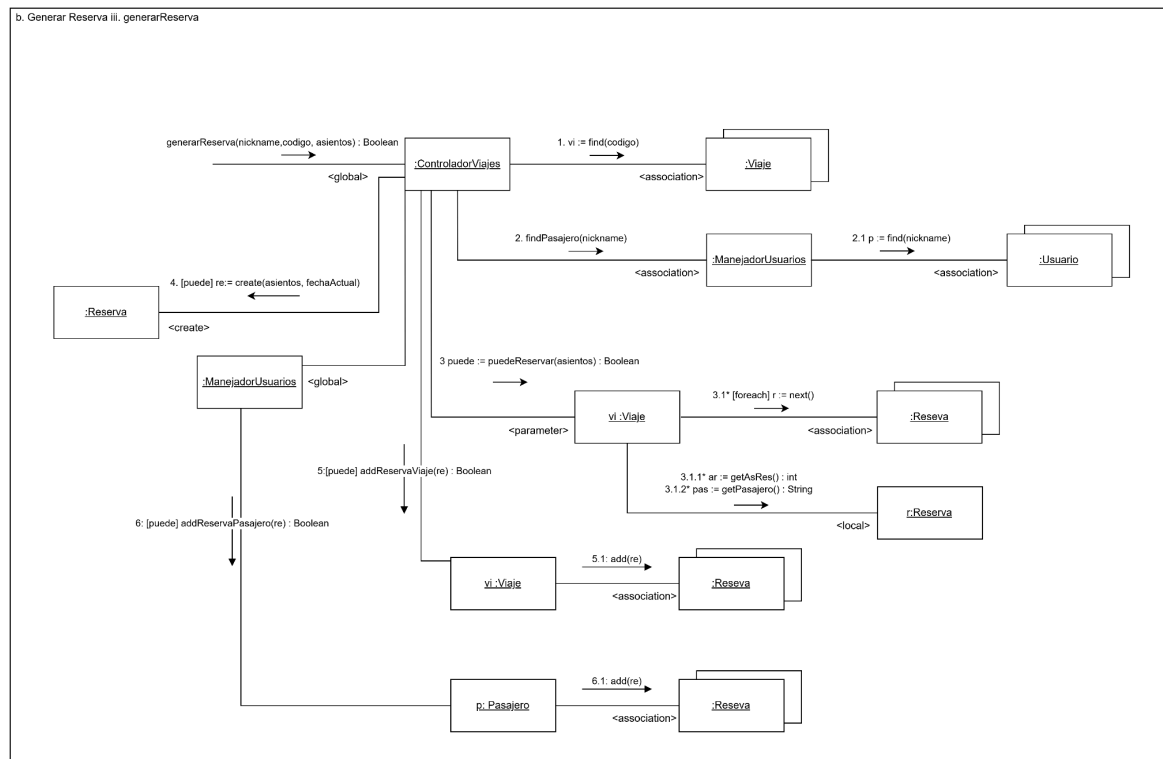


Figura 1.7: Diagrama de comunicación para generar una reserva.

1.2.7 Diagramas de comunicación de listarUsuarios

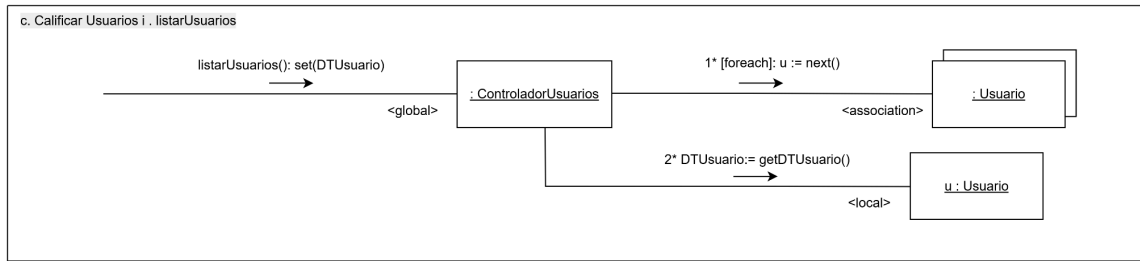


Figura 1.8: Diagrama de comunicación para listar los usuarios existentes.

1.2.8 Diagramas de comunicación de listarViajes

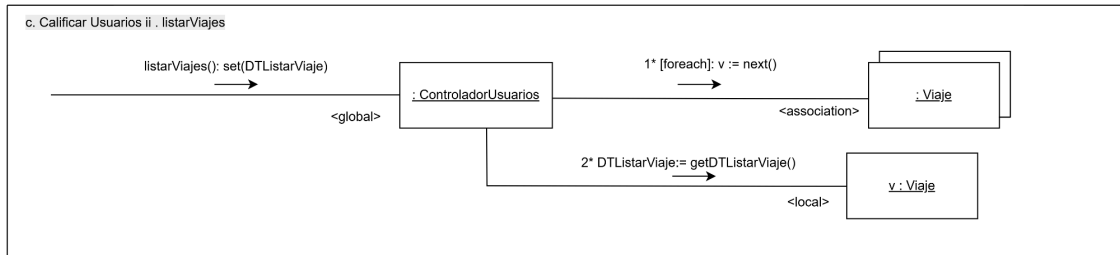


Figura 1.9: Diagrama de comunicación para listar los viajes existentes.

1.2.9 Diagramas de comunicación de listarUsuariosViaje

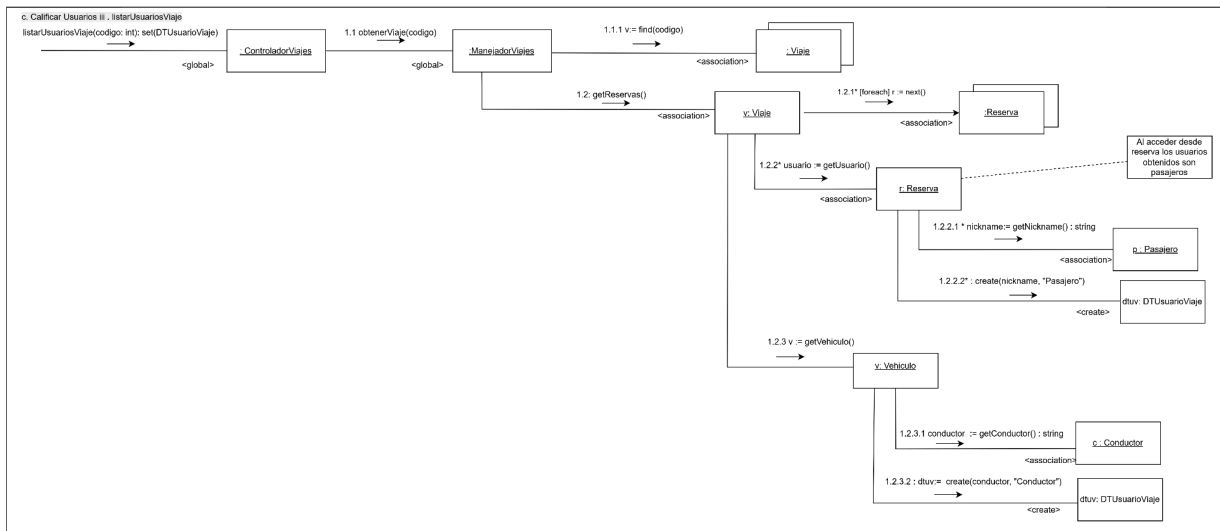


Figura 1.10: Diagrama de comunicación para listar los usuarios de un viaje existente.

1.2.10 Diagramas de comunicación de calificarUsuario

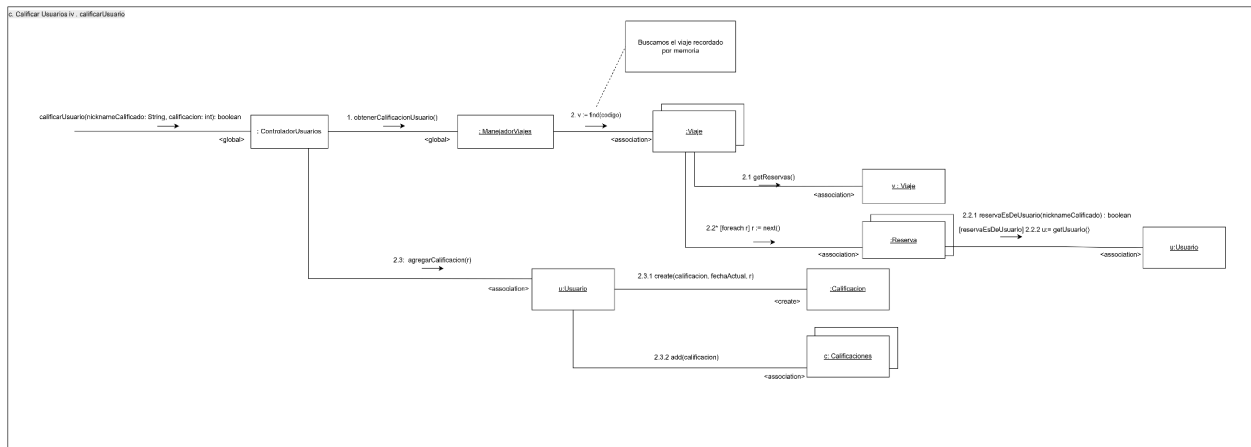


Figura 1.11: Diagrama de comunicación para calificar un usuario.

2 Criterios Generales

2.1 Controller

Se optó por controladores de caso de uso agrupados temáticamente, resultando en dos controladores:

- ControladorUsuarios que agrupa las operaciones relacionadas con usuarios y sus actividades.
- ControladorViajes que agrupa las operaciones relacionadas con la gestión de viajes y las reservas.

Un único controlador hubiese tenido demasiadas operaciones, mezclando varias temáticas, por otro lado un controlador por cada caso de uso genera una cantidad excesiva de controladores sin muchas operaciones para cada uno. En un momento tuvimos un controlador de calificaciones que decidimos remover por esa razón. Llegamos a la conclusión de que con estos dos controladores se mantiene una cohesión razonable y una cantidad manejable de operaciones por controlador.

2.2 Expert

Este criterio se utilizó para distribuir responsabilidades entre las clases del dominio, asignando cada operación al objeto que tiene la información necesaria para resolverla.

Por ejemplo, viaje es la clase que se encarga de hacer la función cumpleCriterio(fecha, origen, destino, asientos) y esto es así ya que viaje es quien conoce todos estos datos y sus reservas y es capaz de verificar si hay reservas que cumplen estos parámetros sin pedir información a otros. Otro ejemplo es como la clase Reserva es quien se encarga de reservaEsDeUsuario(nickname) ya que es la propia reserva la clase experta que conoce a su pasajero asociado.

Asignar responsabilidades a sus respectivos expertos nos asegura que los controladores accedan a datos privados de las clases de dominio innecesariamente.

2.3 Creator

Se aplica este criterio para decidir quién es responsable de crear nuevas instancias

ManejadorUsuarios crea Pasajero y Conductor porque es quien registra y quien contiene la colección global de usuarios.

ManejadorVehiculos crea instancias de Vehículo por la misma razón.

ControladorViajes crea las instancias de Reserva, porque es quien tiene la información agregada y coordina la operación.

ManejadorViajes es quien crea instancias de Calificación mediante la función listaUsuariosViaje, porque tiene acceso al viaje y a la reserva sobre la cual se crea la calificación.

En todos estos casos y otros, la clase creadora cumple al menos uno de los criterios de Creator (contiene, registra o es experta en el tipo en cuestión), garantizando bajo acoplamiento en la creación.

2.4 Bajo Acoplamiento

Este criterio se tuvo en cuenta durante todo el diseño.

Los controladores y manejadores son singleton, las clases acceden a ellos mediante getInstance(). Estos tienen visibilidad global, entonces no es necesario guardarlos como atributos, por lo tanto se evita un acoplamiento innecesario.

Se utilizan datatypes para el intercambio de información, en lugar de retornar entidades del dominio como Usuario, Viaje, etc. Esto desacopla la presentación de las clases del dominio.

Mantener bajo acoplamiento permite que cambios en una clase no impliquen cambios al resto del sistema, haciendo el código más escalable y fácil de mantener.

2.5 Alta cohesión

Este criterio se utilizó puntualmente al separar controladores y manejadores, logrando así que las clases no resulten sobrecargadas y evitando que un cambio pueda afectar varios aspectos al mismo tiempo. Se utilizaron tres manejadores distintos (ManejadorUsuarios, ManejadorVehiculos y ManejadorViajes) y dos controladores (ControladorUsuarios y ControladorViajes).

2.6 No hables con extraños

Para reducir el acoplamiento y la dependencia entre clases, nos aseguramos que cada objeto se comunique únicamente con sus colaboradores directos (vecinos). Por ejemplo, en el caso de listarUsuariosViaje los datos del conductor se “piden” directamente al Vehículo y no se atraviesan varias clases (como podrían ser Usuario).